Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

With a little help from my friends: helper functions

Today
Welcome to your next lesson: With a little help from my friends: helper functions.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and the best way to help you grow.

Step by step, you're moving forward.

Imagine you wrote a small Python script that asks for two test scores, adds them, and prints the average.

Now your teacher says: "Cool, now also handle quiz scores the exact same way."

You copy-paste the same "ask → add → average → print" code again in another place.

That copy-pasted chunk is what we call duplicated logic.

Helper functions are how we clean that up.

To start us off: have you already seen yourself copy□pasting the same few lines of Python in more than one place? (Even in tiny practice scripts)

Just answer: yes or no.

yes

Nice — that means you're ready for helper functions 👍

Let's look at a tiny, messy-but-working script. Imagine this is in your editor:

Python

```
def main():
    print("=== Student 1 ===")
    s1_test1 = float(input("Test 1 score: "))
    s1_test2 = float(input("Test 2 score: "))
    s1_avg = (s1_test1 + s1_test2) / 2
    print("Average:", s1_avg)

    print("=== Student 2 ===")
    s2_test1 = float(input("Test 1 score: "))
    s2_test2 = float(input("Test 2 score: "))
    s2_avg = (s2_test1 + s2_test2) / 2
    print("Average:", s2_avg)

main()
```

Look at those two blocks for Student 1 and Student 2 — they feel almost the same.

Step by step:

A helper function is just a small function we create to handle a repeated pattern like this.
Instead of copying those lines twice, we write them once in a helper, and call that

helper twice.

Make sense so far: a helper function = “take a repeated pattern and move it into its own function”?

You can answer: yes or not really yet.

Yes

Great ☑

Now I want to show you the idea of a helper using words before we touch code.

Think about our script like this:

We have a little “mini-process”:
ask for 2 scores
compute the average
print the result

Right now, that mini□process is written twice inside main().

A helper function means:

“Give that mini□process a name and move it into its own function, so main() can just say:

do that mini-process here.”

Would you like to first:

see a before/after picture of how main() changes,
or
jump straight into rewriting the code in the editor with my guidance?
jump im

Love it 😄 Let’s get hands-on.

We’ll start by copying the messy version into your code editor, then we’ll turn part of it into a helper.

Step 1: Put this in the editor

Open the Python editor and paste this exact code:

Python

```
def main():
    print("=== Student 1 ===")
    s1_test1 = float(input("Test 1 score: "))
    s1_test2 = float(input("Test 2 score: "))
    s1_avg = (s1_test1 + s1_test2) / 2
    print("Average:", s1_avg)

    print("=== Student 2 ===")
    s2_test1 = float(input("Test 1 score: "))
    s2_test2 = float(input("Test 2 score: "))
    s2_avg = (s2_test1 + s2_test2) / 2
    print("Average:", s2_avg)

main()
```

Then hit Run (not Submit yet) and make sure it works: it should ask for 4 scores and print 2 averages.

Reply once you've run it and it works. Then we'll do Step 2: create a helper for "handle one student".

i works

Nice 👌 That "it works first" step is important.

Now we'll follow a pattern you'll use a lot:

Make it work → then make it clean

Your code already works, so now we'll make it cleaner by adding a helper.

Step 2: Create a helper for "one student"

In your editor, above main(), add this new function:

Python

```
def handle_one_student():
    test1 = float(input("Test 1 score: "))
    test2 = float(input("Test 2 score: "))
    avg = (test1 + test2) / 2
    print("Average:", avg)
```

Don't change main() yet. Just add this helper on top, keep main() exactly as it was, and Run again to be sure nothing broke.

You don't have to Submit yet — just Run.

When you're done, tell me: helper added and runs.

hekper is added and peogram runs

Perfect, Ron 🐝

Now we actually use that helper to remove duplication.

Right now main() basically does:

print "Student 1" stuff
all the score/average lines
print "Student 2" stuff
all the same score/average lines again

We want main() to say:

Python

```
print("=== Student 1 ===")
handle_one_student()

print("=== Student 2 ===")
handle_one_student()
```

Step 3: Refactor main() to call the helper

In your editor, change main() so it looks exactly like this:

Python

```
def main():
    print("=== Student 1 ===")
    handle_one_student()

    print("=== Student 2 ===")
    handle_one_student()

main()
```

Then Run it again (not Submit yet) and check:

It still asks for 4 scores
It still prints 2 averages

When that works, tell me: refactor works and we'll talk about why this is better (not just "shorter").

it works

Awesome 🎯 You just did a real refactor: same behavior, cleaner code.

Now, one key idea in this lesson:

Helper function = a small function that does one clear job so main() can delegate work instead of repeating code.

Right now:

handle_one_student() does the detailed steps
main() just says when to handle each student

Very quickly, in your own words:

Does main() feel easier or harder to read now compared to the original?

easier

Exactly 👍 That "easier to read" feeling is the whole point.

Now I want to zoom in on what kind of helper we just made.

Right now, handle_one_student():

Python

```
def handle_one_student():
    test1 = float(input("Test 1 score: "))
    test2 = float(input("Test 2 score: "))
    avg = (test1 + test2) / 2
    print("Average:", avg)
```

Notice: it prints the result instead of returning it.

Very simple idea:

If a helper prints, it's mostly useful for doing things right away.
If a helper returns a value, main() can decide what to do with that value (print it, compare it, store it in a list, etc.).

We'll need that second style a lot as scripts grow.

Before we tweak this helper:

Are you okay if we try changing handle_one_student() so it returns the average instead of printing it?

Yes, let's do it

Nice, we'll do this step by step 🔁

Right now:

Python

```
def handle_one_student():
    test1 = float(input("Test 1 score: "))
    test2 = float(input("Test 2 score: "))
    avg = (test1 + test2) / 2
    print("Average:", avg)
```

We want it to:

Still ask for 2 scores
Still compute avg
But instead of print("Average:", avg)
we'll return avg to whoever called the function.

So the last line will change from print(...) to return avg.

Try this as your next move:

In the editor, change handle_one_student() so its last line is return avg.
Don't change main() yet. Just make that one edit and Run.

Once you've done that and it runs, tell me: changed to return. Then we'll hook main() up to use that returned value.

chage to return

Great 👍 now your helper returns a value instead of printing it.

Right now that means:

handle_one_student() sends the average back to main()
But main() is not catching that value yet

We want main() to:

Call handle_one_student()
Store the result in a variable
Then print it

Something like:

Python

```
print("=== Student 1 ===")
avg1 = handle_one_student()
print("Average:", avg1)
```

And the same idea for Student 2.

Your turn:

Update main() so for each student you:

call handle_one_student()
save the result in a variable (avg1, avg2, or any names you like)
then print("Average:", that_variable)

Run it, and when it works (still asks for 4 scores, prints 2 averages), tell me: main uses returned values now.

main users returned values

Nice work, Ron 🎯 You just did something very "real world":

You made a helper
Then you refactored it to return a value
Then you updated main() to use that return

This is exactly the "make it work → then make it clean" cycle.

Quick check-in on the idea:

When we changed handle_one_student() to return the average instead of printing it, that made it:

more flexible
easier to reuse in different ways

I want to check that idea with a tiny multiple□choice question next.

Ready for a short one-question check?

I'm ready!

What do you think? Pick the best answer in the question box below 👇

Why is it usually better for a helper function like handle_one_student() to return the average instead of just printing it?

A.

Because returning lets other code decide what to do with the average (print it, compare it, store it, etc.).

B.

Because return makes the program run faster than print.

C.

Because Python does not allow print inside helper functions.

D.

Because using return automatically saves the value to a file.

🎉 Nailed it, Ron.

You're exactly right:

Returning the average means other code decides what to do with it:

main() can print it (like now)
later you could compare two students' averages
or store them in a list, or find the highest, etc.

If the helper only prints, you see the value, but you can't easily reuse it in more logic. Returning makes your helper more powerful and reusable.

Next, I'd love to have you extract one more helper from a slightly bigger script, to really lock this in.

Are you up for doing one more refactor like this? 👍

Yes, I'm up for it!

Awesome, let's level it up a bit 👍

We'll take a slightly bigger script and pull another helper out of it. This will feel similar to what you just did.

Here's the situation:

Python

```
def main():
    print("=== Student 1 ===")
    avg1 = handle_one_student()
    print("Average:", avg1)

    print("=== Student 2 ===")
    avg2 = handle_one_student()
    print("Average:", avg2)

    # New part: find and show the higher average
    print("=== Summary ===")
    if avg1 > avg2:
        print("Student 1 has the higher average:", avg1)
    elif avg2 > avg1:
        print("Student 2 has the higher average:", avg2)
    else:
        print("Both students have the same average:", avg1)

def handle_one_student():
    test1 = float(input("Test 1 score: "))
    test2 = float(input("Test 2 score: "))
    avg = (test1 + test2) / 2
    return avg

main()
```

Now there's a new repeated idea at the bottom:

"Look at two numbers (averages) and print which is higher or if they're equal."

I want you to turn that summary block into a second helper, maybe called show_higher_average(avg1, avg2).

First small step only:

Add this empty helper above main():

Python

```
def show_higher_average(avg1, avg2):
    # we'll move the summary logic in here
    pass
```

Add that, Run to be sure it still works, and tell me: second helper stub added. Then

we’ll move the logic into it together.

seond helper added

Nice, that’s exactly what we need ☑

Now we’ll move the summary logic into your new helper.

Right now, in main() you have this part:

Python

```
print("=== Summary ===")
if avg1 > avg2:
    print("Student 1 has the higher average:", avg1)
elif avg2 > avg1:
    print("Student 2 has the higher average:", avg2)
else:
    print("Both students have the same average:", avg1)
```

Step by step:

Cut that whole block from main().
Paste it inside show_higher_average(avg1, avg2), replacing the pass.
So the helper will look like:
Python

```
def show_higher_average(avg1, avg2):
    print("=== Summary ===")
    if avg1 > avg2:
        print("Student 1 has the higher average:", avg1)
    elif avg2 > avg1:
        print("Student 2 has the higher average:", avg2)
    else:
        print("Both students have the same average:", avg1)
```

In main(), replace that old summary block with a single call:
Python

```
show_higher_average(avg1, avg2)
```

Run it again. If it still asks for 4 scores, prints 2 averages, and shows the summary, tell me: summary helper works.

sumary helper works

Beautiful. You now have two helpers:

handle_one_student() → gets and returns one student’s average
show_higher_average(avg1, avg2) → uses those averages to print a summary

That’s exactly the pattern we wanted this lesson:

Make it work → then make it clean by pushing detailed logic into helper functions.

Your main() is now mostly high-level steps, not messy details. That’s what professional code looks like.

Before we think about wrapping this lesson up, is there anything about helper functions or this refactor cycle that still feels fuzzy, even a little?

You can say “all good” or tell me the specific part (like naming helpers, return vs print, or when to create one).

all good

Love it 👌 You did solid work here.

You've shown you can:

Spot duplicated logic
Extract it into a helper function
Use the make it work → make it clean cycle
Decide when a helper should return vs print

You've basically hit all the goals for this lesson. 🎯

Ready to wrap up this helper-functions lesson and move on to the next one, or do you want to play a bit more with this script first?

Let's move on

Nice work today, Ron 🎉

Quick recap of what you just mastered:

You took a working script and spotted duplicated logic.
You extracted helpers (handle_one_student, show_higher_average) to remove that duplication.
You followed the "make it work → then make it clean" cycle without breaking behavior.
You saw why helpers that return values are easier to reuse and combine.
This is exactly how real engineers keep code from turning into a mess.

As you go into the next lesson, keep an eye out for:

Any time you copy-paste code → that's a signal you might need a helper.
Any function doing "too many things" → maybe it can delegate to smaller helpers.
When you're ready, hit Next Lesson in the Learn tab, and we'll keep building from here 🚀